

Software Requirements Specification

Francanglais Compiler and Dataset Platform

Group members and matricules:

Name 1 — Matricule 1

Name 2 — Matricule 2

Name 3 — Matricule 3

Version 1.0 — September 19, 2026

Group work document. Replace all red placeholders before submission.

Contents

1	Introduction	3
1.1	Purpose	3
1.2	Scope	3
1.3	Definitions	3
2	Overall Description	3
2.1	Product perspective	3
2.2	User classes	3
2.3	Operating environment	4
3	Functional Requirements	4
4	Non-Functional Requirements	4
5	Data Requirements	5
6	Use Cases	5
7	Constraints, Assumptions, and Limitations	6
8	Traceability	6

1 Introduction

1.1 Purpose

This document specifies the requirements for a Francanglais (Camfranglais) language data and analysis platform. The verified baseline is the CustomTkinter desktop collector in `data_collector/`; the planned compiler pipeline consumes the collected lexical data for tokenization, classification, prediction, and sentence-level analysis.

1.2 Scope

The system shall allow contributors to collect words, phrases, and sentences with French and English glosses, categories, notes, contributor identity, and optional audio. It shall persist the records locally, support browsing, editing, deletion, audio playback, and statistics, and expose a clear integration boundary for the compiler pipeline.

1.3 Definitions

Francanglais	Code-mixed language using French, English, and Pidgin features in a Cameroonian context.
Entry	A dataset row containing linguistic metadata and optional audio.
Gloss	A standard-language meaning supplied in French or English.
Collector	The desktop application used to create and maintain entries.
Compiler pipeline	The lexer, classifier, predictor, and parser planned to analyze new input using collected evidence.

2 Overall Description

2.1 Product perspective

The collector is a local desktop application. It uses a CSV file as its primary store and an audio directory for media. The repository also contains a compiled frontend artifact; because backend/compiler source is not tracked in the current snapshot, this SRS treats that integration as an explicit future boundary rather than claiming implemented HTTP behavior.

2.2 User classes

- **Contributor:** enters and annotates linguistic data.
- **Maintainer:** reviews, edits, removes, and monitors data quality.
- **Compiler client:** submits text for future lexical/syntactic analysis.

2.3 Operating environment

The collector runs on Python with CustomTkinter. Optional recording uses `sounddevice`, `soundfile`, and `NumPy`. The project stores UTF-8 CSV data and audio files on the local workstation. The documented build environment is Windows, although the audio playback helper includes Unix fallbacks.

3 Functional Requirements

ID	Requirement
FR-01	The system shall create the dataset file and audio directory when they do not exist.
FR-02	The collector shall accept non-empty Francanglais text and reject an empty submission with a visible warning.
FR-03	The contributor shall select one entry type: word, phrase, or sentence.
FR-04	The contributor shall provide French gloss, English gloss, category, notes, and contributor fields.
FR-05	The system shall assign a unique identifier and ISO-8601 timestamp to every saved entry.
FR-06	The system shall append entries using the defined CSV field order without silently discarding fields.
FR-07	The system shall warn about case-insensitive duplicate text before saving.
FR-08	The contributor shall record microphone audio when the optional audio dependencies and device are available.
FR-09	The contributor shall attach an existing audio file when recording is unavailable or undesired.
FR-10	The system shall copy attached/recorded media into the managed audio directory and store its generated filename.
FR-11	The contributor shall browse and search entries by text, category, or contributor.
FR-12	The maintainer shall edit or delete a selected entry and persist the resulting complete CSV.
FR-13	The system shall play an existing audio file using the audio library or operating-system fallback.
FR-14	The system shall show counts by category and entry type.
FR-15	The compiler boundary shall support tokenization, token classification, parsing, and an analysis result for future integration.
FR-16	Compiler features shall not mutate the source dataset without an explicit data-management operation.

4 Non-Functional Requirements

ID	Requirement
NFR-01	The UI shall remain usable at the documented minimum window size of 760 by 620 pixels.
NFR-02	Dataset text shall be read and written as UTF-8 to preserve French and Camfranglais characters.
NFR-03	A failed audio playback operation shall return a visible failure result rather than reporting success.
NFR-04	File paths shall be derived from the project/data directories; user-supplied media shall be copied rather than referenced in place.
NFR-05	The implementation shall keep optional audio support degradable: attach, browse, and edit remain available when recording dependencies fail.
NFR-06	The compiler shall produce deterministic results for the same dataset, grammar, and input.
NFR-07	Data-management operations shall be covered by automated tests for empty input, duplicate handling, CRUD, and CSV round trips.
NFR-08	Secrets and local environment files shall not be included in documentation output or version control.

5 Data Requirements

Each row shall contain the fields `id`, `text`, `entry_type`, `french_gloss`, `english_gloss`, `category`, `notes`, `audio_filename`, `contributor`, and `timestamp`.

Categories include greeting, everyday, slang, insult/banter, market/money, school/campus, food, family, proverb/expression, and other.

6 Use Cases

ID	Actor	Success scenario
UC-01	Contributor	Enter a valid item, annotate it, optionally add audio, and save it to the dataset.
UC-02	Maintainer	Search for an item, select it, edit fields, and save changes.
UC-03	Maintainer	Select an item, request deletion, and see it removed from the CSV.
UC-04	Contributor	Select an audio item and play it through the available playback path.
UC-05	Maintainer	Open Stats and inspect counts by category and entry type.
UC-06	Compiler client	Submit text to the compiler boundary and receive tokens, classifications, and parse analysis.

7 Constraints, Assumptions, and Limitations

- The current verified persistence mechanism is CSV; concurrent multi-user writes are out of scope.
- Audio quality, microphone permissions, and codec support depend on the workstation.
- The teaching grammar in `data_collector/coursework/project.json` is explicitly illustrative and must not be presented as a validated corpus grammar.
- Backend/compiler modules observed only as local ignored artifacts are not treated as released requirements.
- Collected audio and dataset rows are local project data and require a backup policy outside this document.

8 Traceability

The SDD sections and diagrams map FR-01–FR-14 to the collector classes and FR-15–FR-16 to the compiler boundary. The sequence and activity diagrams describe the verified save flow; the component and deployment diagrams separate implemented components from optional/future integration.